UNIVERSIDAD TECNOLÓGICA DE MÉXICO

CAMPUS SUR

MATERIA: SISTEMAS INTELIGENTES

PROYECTO SIMÓN DICE

PROFESOR: MIGUEL ANGEL AGUILAR CRUZ

ALUMNO: ALAN DANIEL ROJAS SERRANO

FECHA DE ENTREGA: 07/04/2026

1. Resumen

Este proyecto consiste en el desarrollo de un sistema interactivo tipo “Simón dice”, en el cual el usuario puede interactuar mediante comandos de voz. El sistema utiliza un microcontrolador ESP32-S3 para el control del hardware, incluyendo una pantalla TFT y un sistema de audio, mientras que el reconocimiento de voz se realiza en una computadora mediante la librería Vosk en Python.

El sistema detecta colores mencionados por el usuario, los procesa y los envía al ESP32 mediante comunicación serial para validar secuencias y generar retroalimentación visual y auditiva.

2. Objetivo

Desarrollar un sistema interactivo que combine reconocimiento de voz y hardware embebido para implementar un juego dinámico que responda a comandos hablados por el usuario.

3. Marco Teórico

Comunicación I2S

El protocolo I2S (Inter-IC Sound) permite la transmisión de audio digital entre dispositivos como micrófonos y microcontroladores. A diferencia del audio analógico, I2S transmite datos en forma digital, lo que reduce el ruido y mejora la calidad de la señal. En este proyecto se utiliza con el micrófono INMP441 y el amplificador MAX98357A.

Comunicación UART

UART (Universal Asynchronous Receiver-Transmitter) es un protocolo de comunicación serial utilizado para transmitir datos entre dispositivos. En este proyecto se utiliza una velocidad de 2,000,000 baudios para permitir el envío rápido de datos de audio sin pérdidas ni retrasos significativos.

Reconocimiento de voz

El reconocimiento de voz permite convertir señales de audio en texto. Se utiliza la librería Vosk, que funciona de manera offline mediante modelos de lenguaje previamente entrenados. Un modelo de lenguaje es un sistema que predice palabras basándose en patrones del lenguaje humano.

Comunicación SPI

SPI es un protocolo de comunicación rápida utilizado para controlar dispositivos como pantallas. En este proyecto se usa para manejar la pantalla TFT ST7789, permitiendo mostrar colores e información visual.

4. Desarrollo Técnico

Hardware

Componentes utilizados

ESP32: Microcontrolador principal encargado de la comunicación y control del sistema.
Micrófono INMP441 (I2S): Captura la señal de audio en formato digital.
Amplificador MAX98357A (I2S): Permite la salida de audio hacia una bocina.
Pantalla TFT ST7789 (SPI): Muestra información visual como colores y estados del sistema.
Bocina 8 ohm 1 W: Reproduce tonos o respuestas auditivas.

Código y conexión para ESP-32 S3 y pantalla ST7789

ST7789	ESP-32 S3
VCC	3.3V
GND	GND
SCL/SCK	GPIO 12
SDA/MOSI	GPIO 11
RES/RST	GPIO 9
DC	GPIO 10
CS	GPIO 8
BLK	3.3V

Código y conexión para ESP-32 S3 y micrófono INMP441

INMP441	ESP-32 S3
VCC	3.3V
GND	GND
WS	GPIO 17
SD	GPIO 15
SCK	GPIO 18
L/R	GND

Código y conexión para ESP-32 S3 y amplificador de audio MAX98357A

MAX98357A	ESP-32 S3
VCC	3.3V
GND	GND
BCLK	GPIO 5
LRC	GPIO 6
DIN	GPIO 7

Software

El proyecto cuenta con dos componentes de software principales: el del ESP32 y el programa de procesamiento de voz en Python. Ambos trabajan en conjunto para lograr el reconocimiento y ejecución de comandos de voz.

ESP32 – S3

1. Librerías, definición de pines y constantes globales

Adafruit_GFX.h

Es la librería de gráficos. Contiene las funciones básicas para dibujar en cualquier pantalla, como crear líneas, círculos, rectángulos y gestionar el tamaño o color del texto.

Adafruit_ST7789.h

Es el controlador específico de la pantalla. Esta librería permite la comunicación con el controlador ST7789 de la pantalla TFT.

SPI.h

Gestiona el protocolo de comunicación Serial Peripheral Interface. Es la vía de alta velocidad que utiliza el ESP32 para enviar los datos de imagen a la pantalla.

driver/i2s.h

Es la librería oficial de Espressif para manejar el protocolo I2S. Se encarga de toda la configuración técnica del sonido: activa el micrófono para "escuchar" y el altavoz para "reproducir".

math.h

Se utiliza para realizar operaciones matemáticas avanzadas. Es fundamental para la función reproducirTono(), ya que permite calcular la función seno necesaria para generar las ondas sonoras que el altavoz convierte en música.

```
#include <Adafruit_GFX.h>
#include <Adafruit_ST7789.h>
#include <SPI.h>
#include <driver/i2s.h>
#include <math.h>
```

Definición de pines de la pantalla

```
#define TFT_CS   8
#define TFT_DC   10
#define TFT_RST  9
#define TFT_SCLK 12
#define TFT_MOSI 11
```

```
Adafruit_ST7789 tft = Adafruit_ST7789(TFT_CS, TFT_DC, TFT_RST);
```

Definición de pines para micrófono usando I2S, frecuencia de muestreo y el tamaño de buffer

```
#define I2S_WS  17
#define I2S_SD  15
#define I2S_SCK 18
#define I2S_PORT     I2S_NUM_0
#define SAMPLE_RATE  16000
#define BUFFER_SIZE  1024
int16_t buffer[BUFFER_SIZE];
```

Definición de pines para altavoz MAX98357A usando I2S y frecuencia de muestreo

```
#define SPEAKER_BCLK  5
#define SPEAKER_LRC   6
#define SPEAKER_DIN   7
#define SPEAKER_PORT  I2S_NUM_1
#define SPEAKER_RATE  16000
```

Definición de variables constantes para secuencia máxima y manejo de nivel

```
#define MAX_SEQ 5
int secuencia[MAX_SEQ];
int nivel = 1;
```

Dimensiones de la pantalla, divididas a la mitad para manejar 4 cuadrantes con mayor facilidad

```
int w = 120;
int h = 160;
```

2. Gestión de audio

Micrófono

Setup de micrófono

Configura e inicializa la interfaz I2S del ESP32 para la recepción de audio digital.

- mode establece el ESP32 como maestro y en modo recepción (RX), lo que significa que genera la señal de reloj y recibe datos de audio.
- sample_rate define la frecuencia de muestreo del audio
- bits_per_sample especifica una resolución de 16 bits por muestra de audio.
- channel_format se utiliza únicamente el canal izquierdo.

```
i2s_config_t i2s_config = {
 .mode = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_RX),
 .sample_rate = SAMPLE_RATE,
 .bits_per_sample = I2S_BITS_PER_SAMPLE_16BIT,
```

```
  .channel_format = I2S_CHANNEL_FMT_ONLY_LEFT,
  .communication_format = I2S_COMM_FORMAT_STAND_I2S,
  .intr_alloc_flags = 0,
  .dma_buf_count = 4,
  .dma_buf_len = 256,
  .use_apll = false
 };

 i2s_pin_config_t pin_config = {
  .bck_io_num = I2S_SCK,
  .ws_io_num = I2S_WS,
  .data_out_num = -1,
  .data_in_num = I2S_SD
 };

 i2s_driver_install(I2S_PORT, &i2s_config, 0, NULL);
 i2s_set_pin(I2S_PORT, &pin_config);
```

Función leerVoz

Esta función captura sonido del micrófono en tiempo real y lo envía a la computadora por el puerto de serie. El programa se mantiene en pausa hasta que recibe un “número” y devuelve el número marcado para ejecutar una acción.

```
int leerVoz() {
 while (true) {
  size_t bytesRead;
  i2s_read(I2S_PORT, buffer, sizeof(buffer), &bytesRead, 1);
  Serial.write((uint8_t*)buffer, bytesRead);

  if (Serial.available()) {
   char comando = Serial.read();

   if (comando == '0') return 0;
   if (comando == '1') return 1;
   if (comando == '2') return 2;
   if (comando == '3') return 3;
  }
 }
}
```

Altavoz

Setup de altavoz

Configura e inicializa la interfaz I2S del ESP32 para reproducir audio digital

- mode establece el ESP32 como maestro y en modo transmisión (TX), lo que significa que genera la señal de reloj y envía datos de audio.

- sample_rate define la frecuencia de muestreo del audio de salida
- bits_per_sample especifica una resolución de 16 bits por muestra de audio.
- channel_format se utiliza únicamente el canal izquierdo.

```
void setupAltavoz() {
 i2s_config_t spk_config = {
  .mode = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_TX),
  .sample_rate = SPEAKER_RATE,
  .bits_per_sample = I2S_BITS_PER_SAMPLE_16BIT,
  .channel_format = I2S_CHANNEL_FMT_ONLY_LEFT,
  .communication_format = I2S_COMM_FORMAT_STAND_I2S,
  .intr_alloc_flags = ESP_INTR_FLAG_LEVEL1,
  .dma_buf_count = 8,
  .dma_buf_len = 256,
  .use_apll = false,
  .tx_desc_auto_clear = true
 };

 i2s_pin_config_t spk_pins = {
  .bck_io_num   = SPEAKER_BCLK,
  .ws_io_num    = SPEAKER_LRC,
  .data_out_num = SPEAKER_DIN,
  .data_in_num  = -1
 };

 i2s_driver_install(SPEAKER_PORT, &spk_config, 0, NULL);
 i2s_set_pin(SPEAKER_PORT, &spk_pins);
}
```

Función reproducirTono

Esta función calcula una onda senoidal basada en la frecuencia y duración y envía los datos al altavoz a través de protocolo I2S para reproducir un tono.

```
void reproducirTono(int frecuencia, int duracionMs) {
 int totalMuestras = (SPEAKER_RATE * duracionMs) / 1000;
 int16_t muestra[256];
 int escritas = 0;
 float amplitud = 8000.0f;

 while (escritas < totalMuestras) {
  int bloque = min(256, totalMuestras - escritas);

  for (int i = 0; i < bloque; i++) {
   float t = (float)(escritas + i) / SPEAKER_RATE;
   muestra[i] = (int16_t)(amplitud * sinf(2.0f * PI * frecuencia * t));
  }
```

```
  size_t bytesEscritos;
  i2s_write(SPEAKER_PORT, muestra, bloque * sizeof(int16_t), &bytesEscritos,
portMAX_DELAY);

  escritas += bloque;
 }
}
```

Función tonoColor

Esta función recibe un número de “color” y lo asocia a una frecuencia, en este caso se usa la escala musical Do, Re, Mi y Sol. Una vez seleccionada la nota, llama a la función reproducirTono para emitir sonido permitiendo que cada color tenga un sonido asociado.

```
void tonoColor(int color) {
 int frecuencias[] = {262, 294, 330, 392};
 reproducirTono(frecuencias[color], 300);
}
```

3. Interfaz Grafica

Función dibujarBase

Divide la pantalla en cuatro rectángulos de colores para crear la interfaz principal del juego.

```
void dibujarBase() {
 tft.fillRect(0, 0, w, h, ST77XX_GREEN);
 tft.fillRect(w, 0, w, h, ST77XX_RED);
 tft.fillRect(0, h, w, h, ST77XX_YELLOW);
 tft.fillRect(w, h, w, h, ST77XX_BLUE);
}
```

Función iluminar

Esta función resalta momentáneamente un color especifico llenando la pantalla con el además de llamar la función tonoColor y luego restaura al tablero original.

```
void iluminar(int color) {
 uint16_t c;

 if (color == 0) c = ST77XX_GREEN;
 if (color == 1) c = ST77XX_RED;
 if (color == 2) c = ST77XX_YELLOW;
 if (color == 3) c = ST77XX_BLUE;

 tft.fillScreen(c);
```

```
 tonoColor(color);
 delay(350);

 dibujarBase();
 delay(150);
}
```

Función mostrarPerder

Muestra una animación de parpadeo con el texto "PERDISTE", reproduce una secuencia de tonos descendentes y reinicia la pantalla al estado inicial.

```
void mostrarPerder() {
 tft.setRotation(1);

 for (int i = 0; i < 6; i++) {
  tft.fillScreen(ST77XX_RED);

  tft.setTextSize(5);
  tft.setTextColor(ST77XX_WHITE);
  tft.setCursor(30, 100);
  tft.print("PERDISTE");

  delay(300);

  tft.fillScreen(ST77XX_BLACK);
  delay(200);
 }

 reproducirTono(400, 300);
 delay(100);
 reproducirTono(300, 300);
 delay(100);
 reproducirTono(200, 500);

 tft.setRotation(0);
 dibujarBase();
}
```

Función mostrarGanar

Activa la animación festiva de colores en pantalla y reproduce una melodía ascendente de tres notas para celebrar la victoria del jugador.

```
void mostrarGanar() {
 tft.setRotation(1);
 loopColores();
```

```
 for (int i = 0; i < 3; i++) {
  reproducirTono(523, 200);
  delay(200);
  reproducirTono(659, 200);
  delay(200);
  reproducirTono(784, 200);
  delay(300);
 }
}
```

Función loopColores

Esta es una función auxiliar de mostrarGanar, realiza un ciclo visual de doce cambios donde el texto "GANASTE!" alterna rápidamente entre diferentes colores brillantes sobre un fondo negro.

```
void loopColores() {
 int j = 0;

 uint16_t colores[] = {
  ST77XX_GREEN,
  ST77XX_RED,
  ST77XX_BLUE,
  ST77XX_YELLOW,
  ST77XX_CYAN,
  ST77XX_MAGENTA
 };

 while (j < 12) {
  for (int i = 0; i < 6; i++) {
   tft.fillScreen(ST77XX_BLACK);

   tft.setTextSize(6);
   tft.setTextColor(colores[i]);
   tft.setCursor(20, 100);
   tft.print("GANASTE!");

   delay(250);
   j++;
  }
 }
}
```

4. Funciones lógicas del juego

Función mostrarSecuencia

Recorre la lista de colores guardada y los va iluminando uno por uno en la pantalla con el objetivo de que el usuario memorice la secuencia.

```
void mostrarSecuencia() {
 for (int i = 0; i < nivel; i++) {
  iluminar(secuencia[i]);
 }
}
```

Función turnoJugador

Procesa los comandos de voz del usuario, ilumina el color que el jugador eligió y verifica si coincide con la secuencia correcta, devolviendo "falso" si hay un error o "verdadero" si completa el nivel con éxito.

```
bool turnoJugador() {
 for (int i = 0; i < nivel; i++) {
  int voz = leerVoz();

  iluminar(voz);

  if (voz != secuencia[i]) return false;

  delay(200);
 }

 return true;
}
```

5. Loop principal

Controla el flujo principal de una partida del juego. Reinicia la secuencia con colores aleatorios, establece el nivel inicial y entra en un bucle donde muestra la secuencia al jugador y espera su respuesta; si el jugador falla, activa la animación de derrota y termina la partida, pero si acierta todos los niveles hasta el máximo, activa la animación de victoria, finalizando el ciclo para reiniciar la ejecución del juego.

```
void loop() {
 for (int i = 0; i < MAX_SEQ; i++) {
  secuencia[i] = random(0, 4);
 }

 nivel = 1;

 while (true) {
  mostrarSecuencia();

  if (!turnoJugador()) {
```

```
    delay(1000);
    mostrarPerder();
    delay(1000);
    break;
   }

   if (nivel == MAX_SEQ) {
    delay(1000);
    mostrarGanar();
    delay(1000);
    tft.setRotation(0);
    break;
   }

   nivel++;
   delay(800);
  }
}
```

Python
El programa en Python se encarga del procesamiento de audio, reconocimiento de voz y envío de comandos al ESP32, utilizando diversas librerías especializadas.

1. Librerías utilizadas y configuración global

serial
Se utiliza para establecer la comunicación UART con el ESP32, permitiendo recibir datos de audio y enviar comandos procesados.

NumPy
Permite manipular los datos de audio de forma eficiente, realizando la conversión desde bytes y aplicando amplificación digital mediante el factor GAIN.

vosk (Model, KaldiRecognizer)
Se utiliza para el reconocimiento de voz offline. El modelo carga un conjunto de datos preentrenados y el reconocedor procesa el audio para convertirlo en texto.

json
Permite interpretar las respuestas generadas por Vosk, extrayendo el texto reconocido a partir de estructuras en formato JSON.

difflib (get_close_matches)
Se utiliza para corregir palabras reconocidas mediante coincidencias aproximadas, mejorando la precisión del sistema.

También se definen parámetros clave como el puerto serial que se va a utilizar, la frecuencia de muestreo, ganancia de audio y la velocidad de comunicación a la que se envían los datos.

```
import serial
import numpy as np
from vosk import Model, KaldiRecognizer
import json
from difflib import get_close_matches

PORT = 'COM3'
BAUD = 2000000
SAMPLE_RATE = 16000
GAIN = 5
```

Se declara la variable serial la cual se usa para establecer comunicación con el ESP32 para recibir audio y enviar comandos

```
ser = serial.Serial(PORT, BAUD)
```

Se carga el modelo de voz en español de vozk e inicializa el reconocedor con frecuencia de 16 kHz

```
model = Model("vosk-model-small-es-0.42")
rec = KaldiRecognizer(model, SAMPLE_RATE)
```

Se define las palabras que se esperan recibir y se mapean para enviar un número en código binario cuando se reconozca un color.

```
COLORES = ["rojo", "verde", "azul", "amarillo"]
COLOR_CMD = {
    "verde": b'0',
    "rojo": b'1',
    "amarillo": b'2',
    "azul": b'3'
}
```

Función corregir_color

Esta función toma una palabra capturada por el micrófono y si el modelo de voz no la entendió perfectamente busca la coincidencia más cercana dentro de la lista de colores.

```
def corregir_color(palabra):
    match = get_close_matches(palabra, COLORES, n=1, cutoff=0.6)
    return match[0] if match else None
```

Captura y limpieza de audio

Este bloque de codigo lee los datos binarios que llegan del ESP32, los datos se convierten a numeros con numpy para multiplicar su volumen con GAIN mientras se asegura que no se pasen los limites. Finalmente los datos se convierten a formato de audio para que el modelo los entienda.

```
while True:
 data = ser.read(1024)

 # convertir a numpy
 audio_np = np.frombuffer(data, dtype=np.int16)
```

```
# amplificar
audio_np = audio_np * GAIN
audio_np = np.clip(audio_np, -32768, 32767)

# regresar a bytes
processed = audio_np.astype(np.int16).tobytes()
```

Reconocimiento de voz

Se envía el audio amplificado al motor de vozk, AcceptWaveForm devuelve verdadero cuando detecta silencio, en ese momento se extrae el texto del archivo json generado.

```
if rec.AcceptWaveform(processed):
 result = json.loads(rec.Result())
 texto = result.get("text", "")
```

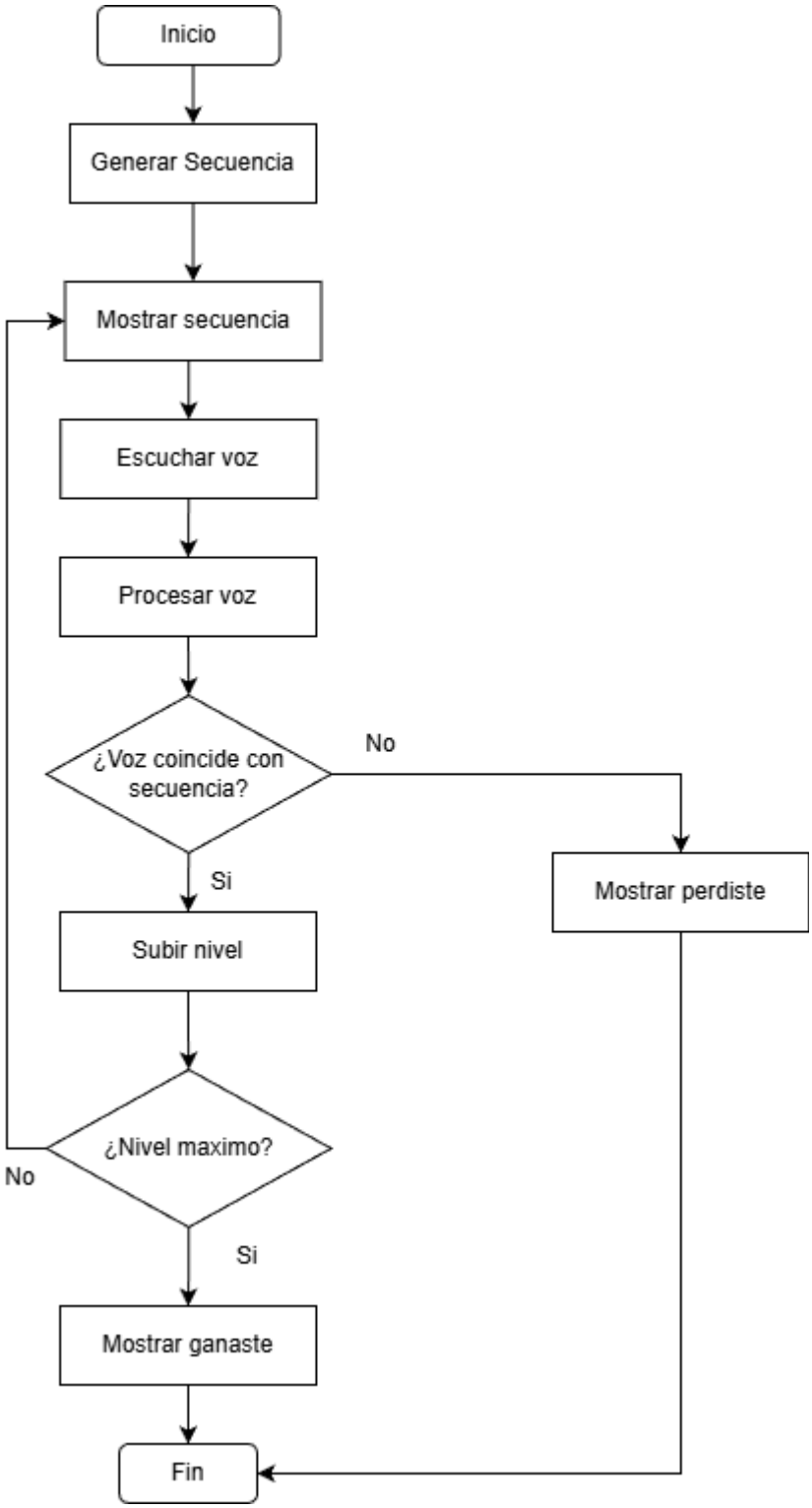
Si el modelo detecta entrada de voz separa la frase en palabras individuales y revisa cada palabra con la función corregir_color para identificar coincidencias con los colores definidos y guarda las coincidencias encontradas junto a su posición en la frase detectada.

```
if texto:
 print("Texto final:", texto)

colores_detectados = []
palabras = texto.split()

 for i, palabra in enumerate(palabras):
 color_corregido = corregir_color(palabra)

 if color_corregido:
 colores_detectados.append((i, color_corregido))
```

Se ordenan los colores en el orden de la frase y luego se busca el valor binario correspondiente en el diccionario y se envía al ESP32 para que el juego registre la jugada.

```
colores_detectados.sort()

for _, color in colores_detectados:
 print("Enviando color al esp32:", color)
 comando = COLOR_CMD[color]
 ser.write(comando)
```

Diagrama de flujo



Referencias

- ESPHome. (s.f.). I²S Audio Microphone. ESPHome Documentation. Recuperado el 3 de abril de 2026, de Microphone https://esphome.io/components/microphone/i2s_audio/

- Alpha Cephei Inc. (2023). Vosk API documentation. Recuperado el 3 de abril de 2026, de https://alphacephei.com/vosk/

- Python Software Foundation. (2024). pySerial documentation. https://pyserial.readthedocs.io/

- Espressif Systems. (2023). ESP32 Technical Reference Manual. https://www.espressif.com/